

Aesopic Inc. - Fall 2026 Vision Agents Co-op

Approach

For the core architecture, I used an observe → decide → act loop:

1. Screenshot viewport
2. Send PNG, goal, URL, and recent action history to vision model (via Ollama API/Gemini API)
3. Parse one JSON action (`click`, `type`, `scroll`, `wait`, `press_key`, or `done`)
4. Execute via Playwright wrapper
5. Repeat until done or limits hit (420s time limit or 30 step limit)

The CLI (`navigate.py`) wires configuration; `agent/loop.py` orchestrates the loop; `agent/browser.py` exposes a minimal Playwright surface (`click` at normalized coordinates, `type`, `scroll`, etc.); and `agent/prompts.py` encodes GitHub-specific heuristics because those cannot live in selector code.

Below are the key design choices I made as well as their rationales:

Choice	Rationale
Normalized coordinates (0–1000)	Model outputs viewport-relative coords; Playwright maps to pixels with clamping.
One action per turn	Simplifies parsing, debugging, and recovery.
Structured actions via Pydantic	Strict validation; discriminated union for action types; <code>done</code> carries all output fields.
Prompt-driven navigation	No hardcoded GitHub script; flow emerges from system and user prompt.
Backup backend vision model (Gemini API)	Ollama Cloud (<code>qwen3-vl:235b-cloud</code>) primary; Gemini optional fallback via <code>create_vision_client()</code> . Fallback helped avoid rate limits while working with free-tier models.

Work shipped in three milestones: v1 (five core fields + proof of navigation), v2 (release notes, publish date, download assets), v3 (JSON normalization, code comments, committed OpenClaw sample artifact). Each milestone had explicit acceptance criteria and ad-hoc E2E validation rather than a formal multi-repo test matrix.

What Worked

Vision-only navigation proved sufficient for structured GitHub release extraction; verified on `facebook/react` (9 steps, all eight fields) and `openclaw/openclaw` (committed `openclaw-release.json`). Most failures were recoverable through prompt tuning and runtime guardrails (sidebar mis-clicks, wrong release selection, scroll loops) rather than DOM selectors. Lenient JSON parsing was equally critical: Ollama frequently returned truncated or malformed

JSON, and the repair pipeline kept runs completing despite intermittent parse warnings. For repos with very long changelogs, a “partial notes + End key for Assets” workflow replaced exhaustive scrolling and stopped max-step exhaustion. --debug step screenshots and 57 unit tests on deterministic paths (parsing, coordinates, validation) made iteration tractable.

What Didn't Work

Gemini (initial model of choice) as primary backend hit rate limits early; Ollama Cloud became the default. Release selection and scrolling required several prompt rewrites: early runs returned pre-releases, wrong versions, or exhausted the step cap on long changelogs (i.e. openclaw). The initial 180s time budget was too tight for v2's extra read steps plus slow cloud inference (causing a raise to 420s). Ollama JSON flakiness persists as noise even on successful runs; the repair layer mitigates but does not eliminate it. On long changelogs/release notes, partial scraping didn't always scrape the most relevant pieces of info.

Trade-Offs

Decision	Benefit	Cost
Vision-only (no selectors/API)	Layout-resilient; matches take-home intent	Slower, less deterministic than DOM/API
Topmost release = target	Matches “current release” literally	May return pre-release over GitHub's “Latest” stable
Partial release notes OK	Reliable on long changelogs	Not a full changelog copy
Prompt heuristics vs coded flow	Flexible --prompt phrasing	Behavior varies by repo and model

Limitations

These are intentional scope boundaries, not oversights:

- **Release notes and URLs are vision-extracted:** long changelogs are partial; off-screen asset links may be missing (anti-hallucination rules omit guessed URLs).
- **Release selection semantics:** the agent targets the topmost published release, not necessarily GitHub's “Latest” stable label.
- **Operational constraints:** Ollama Cloud session limits and Gemini quotas can block runs; default 420s wall-clock budget may still be tight on slow inference.
- **Fixed output schema:** output is always the eight-field release snapshot; open-ended tasks (compare releases, summarize features) are out of scope.
- **Explicit non-goals:** private repos/auth, captcha, multi-tab handling, GitHub REST/GraphQL fallback.

Future Improvements

1. --repo owner/name — skip search navigation when the target is known.
2. Flexible output schema: let --prompt drive different output shapes (comparison, summary, feature list)
3. Using stronger/efficient policy models (i.e. Local Ollama) for development to avoid quota limits
4. Configurable release policy (topmost vs stable “Latest”)